

P.PORTO

REST APIs

Programação em Ambiente Web

Índice

Introdução

REST API

OpenAPI e Swagger

API Requests

Referências

Introdução

REST (Representational State Transfer)

Roy Fielding definiu REST na sua dissertação de doutoramento em 2000 "Architectural Styles and the Design of Network-based Software Architectures"



Fonte:

https://en.wikipedia.org/wiki/File:Roy_Fielding_at_OSCON_2008.jpg

Introdução

API (Application Program Interface): é um conjunto de funções e procedimentos que permitem a criação de aplicações que acedem a recursos ou dados de um sistema, aplicação ou outro serviço

REST API: definição de apis baseadas no protocolo HTTP universal

Introdução

Um serviço REST, contém um padrão de software arquitetural que define um conjunto de restrições e propriedades baseados em métodos HTTP

Web Services que obedecem ao padrão arquitetural REST, ou web services RESTful, fornecem interoperabilidade entre sistemas na Internet através da representação dos recursos sobre a forma de texto

Existem outro tipo de web services, como web services SOAP, expõem seus próprios conjuntos arbitrários de operações

Introdução

Num web service RESTful, pedidos feitos a um URI de um recurso obterá uma resposta que pode estar em XML, HTML, JSON ou algum outro formato

Os métodos HTTP mais usados em serviços REST, são GET, POST, PUT, DELETE e que predefinem o CRUD em HTTP

Introdução

Os serviços REST proporcionam

- desempenho rápido,
- confiabilidade
- reutilização de componentes

O REST ignora os detalhes da implementação de componentes e a sintaxe de protocolos com o objetivo de se focar

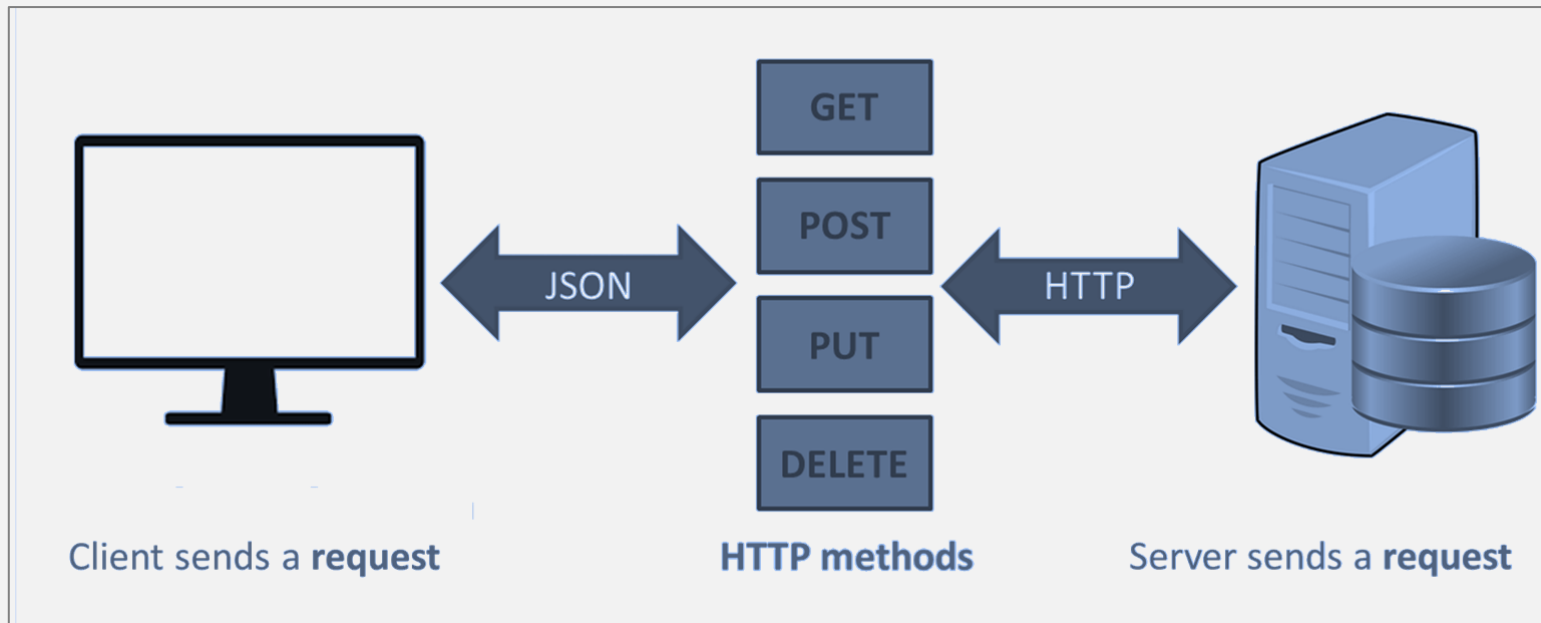
- no papel dos componentes
- nas restrições sobre sua interação com outros componentes

Introdução

Propriedades de serviços REST

- Arquitetura cliente-servidor
- Statelessness
- Cacheability
- Layered system
- Uniform interface

Introdução



REST API

Os quatro métodos HTTP a seguir são comumente usados na arquitetura baseada em REST:

- GET - é usado para fornecer acesso somente leitura a um recurso.
- PUT - é usado para atualizar e para criar um novo recurso.
- DELETE - é usado para remover um recurso.
- POST - é usado para criar um novo recurso.

Introdução

Métodos HTTP comumente utilizados por API REST

Método	Objetivo
GET	Obtêm recursos do servidor
POST	Cria um novo recurso no servidor
PUT	Atualiza um recurso no servidor
DELETE	Elimina um recurso no servidor

REST API

Exemplo em node.js :

```
/* GET product listing. */  
router.get('/products', productController.getAllProducts);  
router.post('/products', productController.createProduct);  
  
router.get('/product/:productId', productController.getOneProduct);  
router.put('/product/:productId', productController.updateProduct);  
router.delete('/product/:productId', productController.deleteProduct);
```

Serviços REST não exigem que o cliente saiba alguma coisa sobre a estrutura da API

O servidor especifica a localização do recurso e os campos obrigatórios. O browser não sabe antecipadamente onde enviar as informações e não sabe de antemão quais informações enviar

REST API

Problema:

- Vamos considerar o problema de gerir o stock de um produto
- Iremos criar um conjunto de serviços web armazenar informação sobre produtos e as suas quantidades

REST API

Antes de começar um projeto que defina uma API do tipo REST devemos refletir sobre o tipo de operações que devemos permitir

De seguida devemos associar os métodos do protocolo HTTP a cada uma dessas ações

Por fim definimos o endereço de cada uma destas ações

REST API

Método HTTP	Path	Input	Output
GET	/products	-	lista de produtos
POST	/products	produto	-
GET	/product/prodID	prodID	Produto
DELETE	/product/prodID	prodID	-
PUT	/product/prodID	/product/prodID	-

REST API

Método HTTP	Path	Input	Output
GET	/products	-	lista de produtos
POST	/products	produto	-
GET	/product/prodID	prodID	Produto
DELETE	/product/prodID	prodID	-
PUT	/product	produto	-



REST API

Podemos agora criar o projeto REST API

Usar o gerador Express:

- “express RESTAPI”

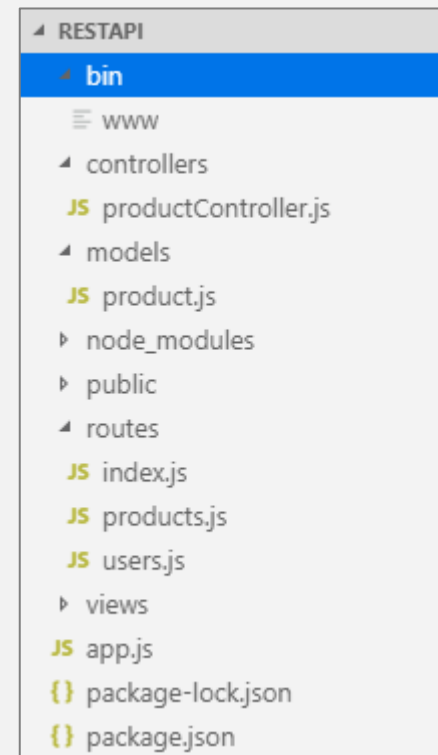
Criar o padrão MVC:

- Criar controladores
- Criar modelos
- As views serão criadas pelo middleware no formato JSON

REST API

O projeto ter a estrutura exemplificada na figura ao lado

Devemos sempre que possível estruturar a aplicação da melhor forma possível



REST API

Criar o ficheiro
product.js na pasta
models

```
var mongoose = require('mongoose');
var Schema = mongoose.Schema;

var ProductSchema = new Schema({
  name: { type: String },
  description: {type: String},
  quantity: {type: Number}
});

module.exports = mongoose.model('Product', ProductSchema);
```

REST API

Criar o ficheiro
productController.js
dentro da pasta
controllers

Definir todas as
ações do
controlador do
produto e a
interação com uma
base de dados em
mongoDB

```
var mongoose = require("mongoose");
var Product = require("../models/product");

var productController = {};

productController.createProduct = function (req, res, next) {
  var product = new Product(req.body);

  product.save(function (err) {
    if (err) {
      next(err);
    } else {
      res.json(product);
    }
  });
};

productController.updateProduct = function (req, res, next) {
  Product.findByIdAndUpdate(req.body._id, req.body, {new: true}, function
  (err, product) {
    if (err) {
      next(err);
    } else {
      res.json(product);
    }
  });
};
```

REST API

Criar o ficheiro
productController.js
dentro da pasta
controllers

Definir todas as ações
do controlador do
produto e a interação
com uma base de
dados em mongoDB

```
productController.deleteProduct = function (req, res, next) {  
  req.product.remove(function (err) {  
    if (err) {  
      next(err);  
    } else {  
      res.json(req.product);  
    }  
  });  
};  
  
productController.getAllProducts = function (req, res, next) {  
  Product.find(function (err, products) {  
    if (err) {  
      next(err);  
    } else {  
      res.json(products);  
    }  
  });  
};  
  
productController.getOneProduct = function (req, res) {  
  res.json(req.product);  
};
```

REST API

Criar o ficheiro
productController.js
dentro da pasta
controllers

Definir todas as acções
do controloador do
produto e a interação
com uma base de dados
em mongoDB

```
productController.getByIdProduct = function (req, res, next, id) {  
  Product.findOne({_id: id}, function (err, product) {  
    if (err) {  
      next(err);  
    } else {  
      req.product = product;  
      next();  
    }  
  });  
};  
  
module.exports = productController;
```

REST API

Criar o ficheiro
product.js
dentro da pasta
routes

Aqui são
definidos os
métodos http
para cada ação

```
var express = require('express');
var router = express.Router();
var productController = require("../controllers/productController.js");

/* GET product listing. */
router.get('/products', productController.getAllProducts);
router.post('/products', productController.createProduct);

router.get('/product/:productId', productController.getOneProduct);
router.put('/product/:productId', productController.updateProduct);
router.delete('/product/:productId', productController.deleteProduct);

router.param('productId', productController.getByIdProduct);

module.exports = router;
```

REST API

Atualizar o
ficheiro app.js
com a
informação
relevante para
servir a API REST

```
var createError = require('http-errors');
var express = require('express');
var path = require('path');
var cookieParser = require('cookie-parser');
var logger = require('morgan');

var productsRouter = require('./routes/products');
var mongoose = require('mongoose');
mongoose.Promise = global.Promise;

mongoose.connect('mongodb://localhost:27017/product-demo')
  .then(() => console.log('connection succesful'))
  .catch((err) => console.error(err));

var app = express();

app.use(logger('dev'));
app.use(express.json());
app.use(express.urlencoded({ extended: false }));
app.use(cookieParser());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/api/v1', productsRouter);
```


REST API

Atualizar o
ficheiro app.js
com a
informação
relevante para
servir a API
REST

```
// catch 404 and forward to error handler
app.use(function(req, res, next) {
  next(createError(404));
});

// error handler
app.use(function(err, req, res, next) {
  // set locals, only providing error in development
  res.locals.message = err.message;
  res.locals.error = req.app.get('env') === 'development' ? err : {};

  // render the error page
  res.status(err.status || 500);
  res.render('error');
});

module.exports = app;
```

REST API

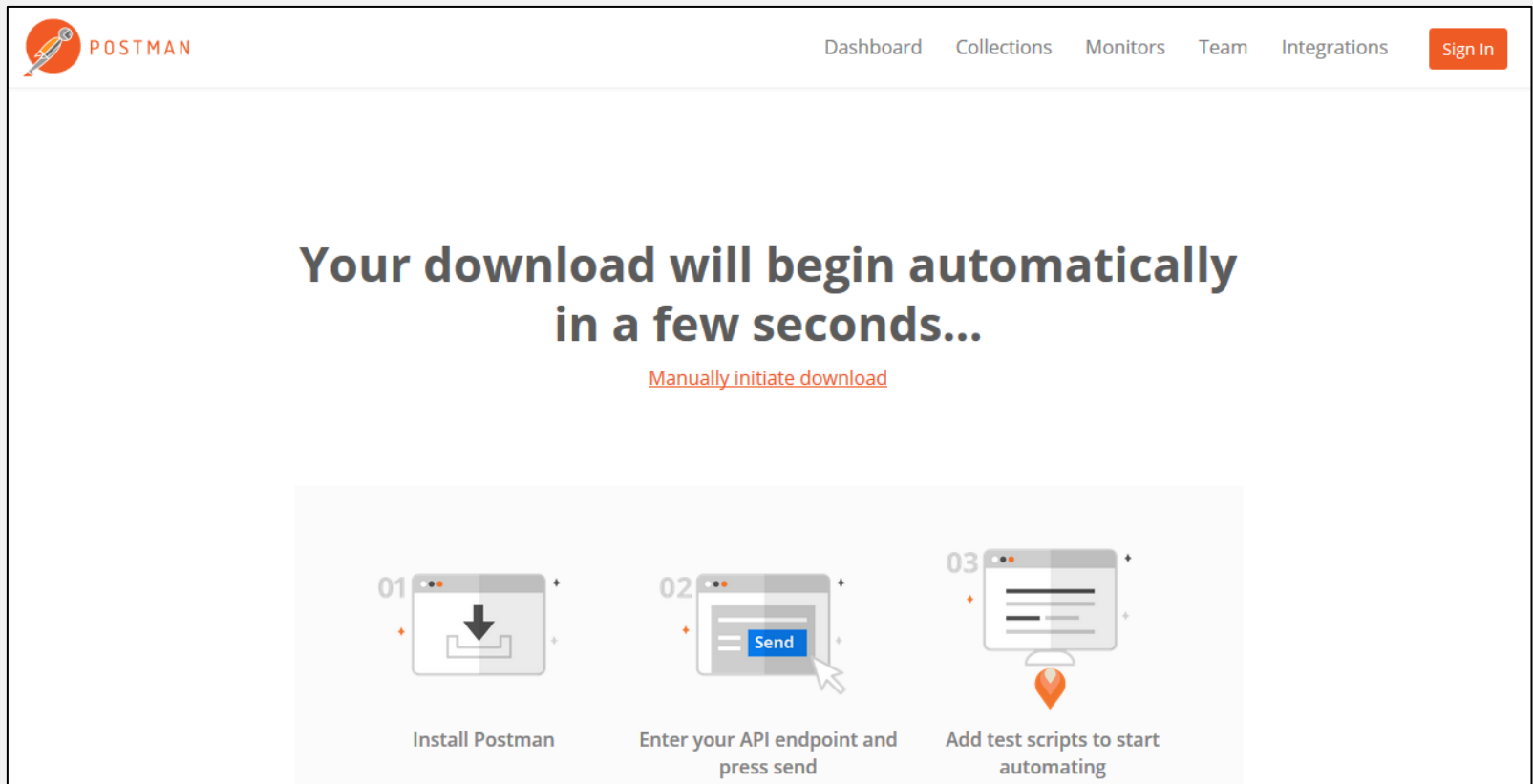
Podemos usar ferramentas externas para o teste da nossa API

Uma das mais conhecidas é a extensão postman que permite:

- Enviar pedidos REST
- Definir parâmetros na área Header e Body dos pedidos HTTP
- Verificar as respostas do servidor
- Criar projetos para teste e validação de APIs

<https://www.getpostman.com/>

REST API



The image is a screenshot of the Postman web application dashboard. At the top left is the Postman logo (an orange circle with a white key icon) and the word "POSTMAN" in orange. To the right of the logo are navigation links: "Dashboard", "Collections", "Monitors", "Team", and "Integrations". On the far right is a red "Sign In" button. The main content area has a large heading "Your download will begin automatically in a few seconds..." in bold black text. Below this heading is a red link "Manually initiate download". At the bottom, there is a light gray box containing a three-step guide for getting started with Postman. Each step is numbered and includes an icon and a description: 01. An icon of a download arrow pointing into a box, with the text "Install Postman". 02. An icon of a "Send" button being clicked, with the text "Enter your API endpoint and press send". 03. An icon of a document with a location pin, with the text "Add test scripts to start automating".

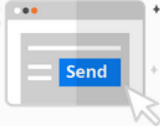
POSTMAN

Dashboard Collections Monitors Team Integrations Sign In

**Your download will begin automatically
in a few seconds...**

[Manually initiate download](#)

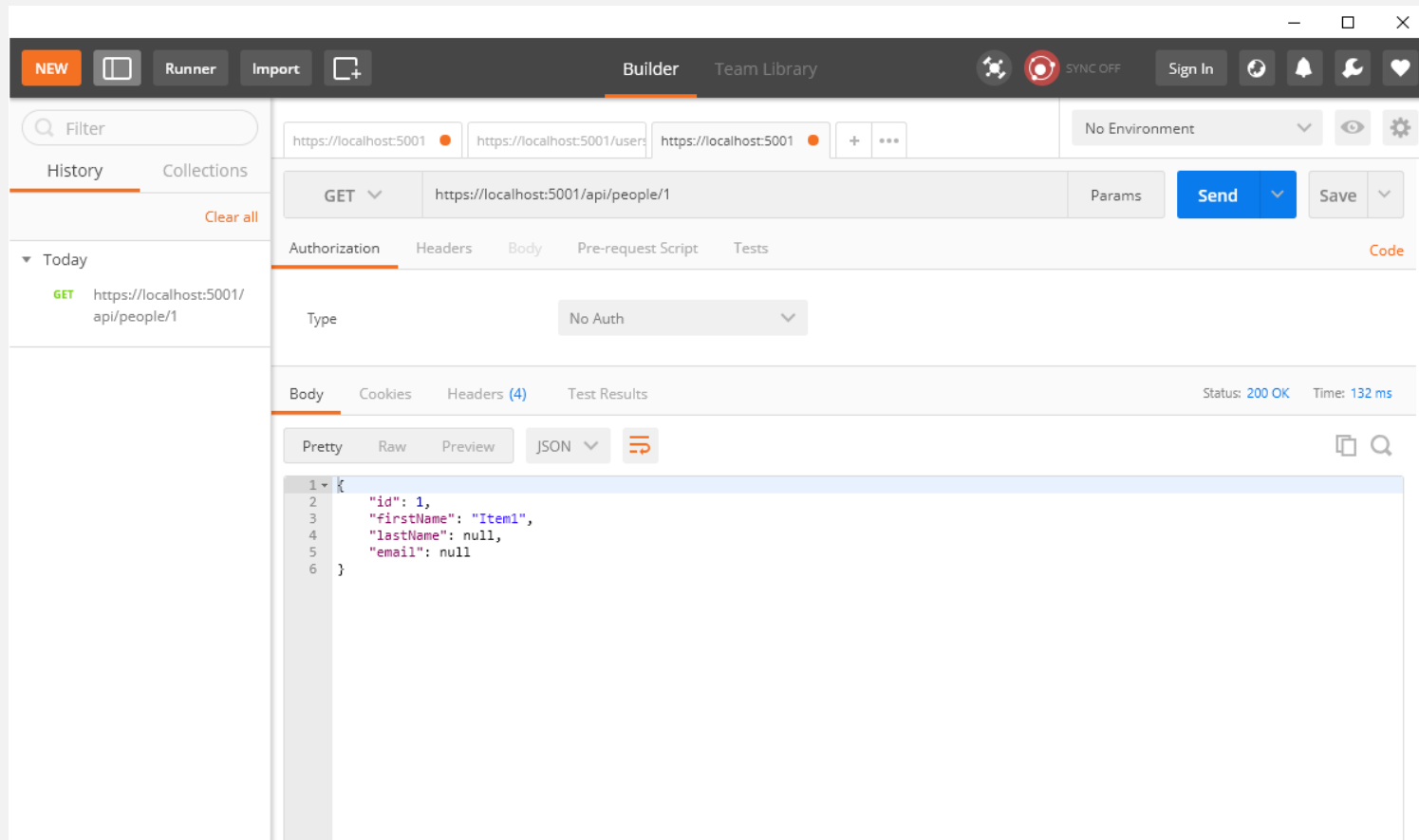
01  Install Postman

02  Enter your API endpoint and
press send

03  Add test scripts to start
automating

Fonte: <https://app.getpostman.com/app/download/win64>

REST API



Teste de
um
pedido
get

REST API

The screenshot displays the REST Client application interface. The top bar includes a 'NEW' button, a 'Runner' button, an 'Import' button, and a 'Builder' tab. The main workspace shows a POST request to `https://localhost:5001/api/people`. The request body is a JSON object: `{ "firstName": "Me", "lastName": "Me too", "email": "Me again" }`. The response is also shown in the bottom panel, with a status of 201 Created and a time of 88 ms. The response body is a JSON object: `{ "id": 2, "firstName": "Me", "lastName": "Me too", "email": "Me again" }`.

History

Today

- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people`
- POST `https://localhost:5001/api/people/1`
- GET `https://localhost:5001/api/people/1`

Builder

Team Library

Sign In

SYNC OFF

No Environment

POST `https://localhost:5001/api/people`

Params

Send

Save

Authorization

Headers (1)

Body

Pre-request Script

Tests

Code

form-data

x-www-form-urlencoded

raw

binary

JSON (application/json)

```
1 {
2   "firstName": "Me",
3   "lastName": "Me too",
4   "email": "Me again"
5 }
```

Body

Cookies

Headers (8)

Test Results

Status: 201 Created

Time: 88 ms

Pretty

Raw

Preview

JSON

```
1 {
2   "id": 2,
3   "firstName": "Me",
4   "lastName": "Me too",
5   "email": "Me again"
6 }
```

Teste de
um pedido
post com
parâmetros
no Body do
request
HTTP

REST API

Demonstração Prática
(Postman)

REST API

Os serviços REST podem ser diretamente acedidos por páginas HTML simples usando pedidos AJAX ao servidor

Contudo é necessário conhecimento sobre a API REST para interagir com serviços REST

Devemos sempre documentar os nossos serviços REST

REST API

Por defeito, os browsers irão bloquear pedidos a servidores com domínio diferente de onde a página foi carregada

Para que os nossos web services REST sejam acessíveis por domínios de outras páginas web é preciso que se definam headers específicos nas mensagens HTTP

REST API

Devemos instalar o package cors com o comando:

- “npm i cors --save”

Dentro do ficheiro app.js

```
var express = require('express');  
var cors = require('cors');  
var app = express();  
  
app.use(cors());
```

REST API

Podemos também apenas aplicar CORS a apenas um único tipo de pedido com funções de middleware diretamente no router em express

```
var express = require('express');
var cors = require('cors');
var app = express();

app.get('/products/:id', cors(), function (req, res, next) {
  res.json({msg: 'This is CORS-enabled for a Single Route'});
});
```

REST API

Uma explicação completa sobre o funcionamento de políticas CORS para estudo futuro pode ser encontrado em:

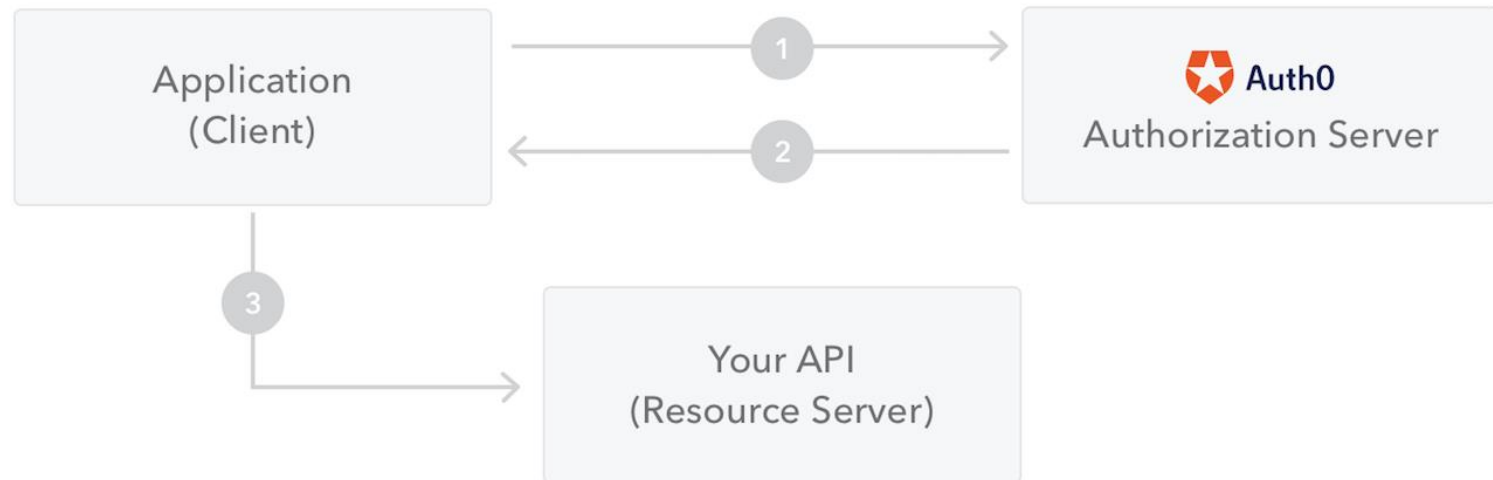
- <https://flaviocopes.com/cors/>

REST API e JWT

JSON Web Token (JWT) é um open standard (RFC 7519) que define um método compacto e autocontido para transmitir com segurança informações entre as partes num objeto JSON.

As JWTs podem ser assinados utilizando um segredo (com o algoritmo HMAC) ou um par de chaves pública/privada usando RSA ou ECDSA.

REST API e JWT



REST API e JWT

Embora JWTs possam ser criptografadas para também fornecer sigilo entre as partes, iremos focar-nos no use de signed tokens

Os signed tokens podem verificar a integridade da informação contidas nele, enquanto os tokens criptografados ocultam essas declarações de outras partes

Quando os tokens são assinados usando pares de chaves pública/privada, a assinatura também certifica que a parte que é proprietária da chave privada é aquela que a assinou

REST API e JWT

Em node.js podemos adicionar a verificação por JWT com o módulo:

- `npm install jsonwebtoken --save`

Devemos criar um controlador para tratar a autenticação e um segredo para criar tokens

REST API e JWT

Controlador para criar a autenticação:

```
// AuthController.js
var express = require('express');
var router = express.Router();
var bodyParser = require('body-parser');
router.use(bodyParser.urlencoded({ extended: false }));
router.use(bodyParser.json());
var User = require('../user/User');
var jwt = require('jsonwebtoken');
var config = require('../config');
router.post('/register', function(req, res) {

  User.create({
    name : req.body.name,
    email : req.body.email,
    password : req.body.password
  },
  function (err, user) {
    if (err) return res.status(500).send("There was a problem registering the user.")
    // create a token
    var token = jwt.sign({ id: user._id }, config.secret, {
      expiresIn: 86400 // expires in 24 hours
    });
    res.status(200).send({ auth: true, token: token });
  });
});

module.exports = router;
```


REST API e JWT

Config.js

```
// config.js
module.exports = {
  'secret': 'supersecret'
};
```

Adicionar a app.js

```
// app.js
var AuthController = require('./auth/AuthController');
app.use('/api/auth', AuthController);
module.exports = app;
```

REST API e JWT

Verificar o acesso a um recurso que necessita de autenticação

```
router.get('/me', function(req, res) {  
  var token = req.headers['x-access-token'];  
  if (!token) return res.status(401).send({ auth: false, message: 'No token  
provided.' });  
  
  jwt.verify(token, config.secret, function(err, decoded) {  
    if (err) return res.status(500).send({ auth: false, message: 'Failed to  
authenticate token.' });  
  
    res.status(200).send(decoded);  
  });  
});
```

Verificar a existência de um token nos header do pedido http e autenticar junto do módulo JWT

OpenAPI e Swagger

OpenAPI e Swagger

- Foram criados por um consórcio do setor que reconhecem o valor da padronização de como as APIs REST são descritas.
- Como uma estrutura de governance aberta sob a Linux Foundation, a OAI está focada na criação, evolução e promoção de um formato de descrição de API REST neutra de fornecedor.
- Criar um formato de descrição aberto para serviços de API que seja neutro, portátil e aberto é importante para melhorar o acesso e interoperabilidade dos sistemas

OpenAPI e Swagger

O openAPI define um conjunto de métodos e boas práticas na definição de webservices que são seguidos pela plataforma swagger

Principais características presentes na plataforma swagger:

Design

Build

Document

Test

Standardize

OpenAPI e Swagger

The screenshot shows the OpenAPI Initiative website. At the top is a navigation bar with the OpenAPI Initiative logo on the left and links for About, Specification, Participate, Membership, Blog, Events, FAQ, and Get Involved on the right. Social media icons for Twitter, LinkedIn, and GitHub are also present. Below the navigation bar is a large banner celebrating the 'HAPPY 3RD BIRTHDAY' of the OpenAPI Initiative. The banner features two LEGO minifigures, one wearing a party hat, holding balloons, and a large '3RD' in the text. The OpenAPI Initiative logo is on the right side of the banner. Below the banner are two buttons: 'How to contribute to the OAS' and 'Submit an issue on GitHub'. At the bottom, there is a row of logos for partner organizations: Tyk.io, ISA, MuleSoft, Intento, ca technologies, and 42crunch. On the right side of the bottom row, there is a box with the text 'JOIN THE GROWING LIST OF OAI MEMBERS'.

OPENAPI INITIATIVE

About Specification Participate Membership Blog Events FAQ Get Involved

Twitter LinkedIn GitHub

HAPPY 3RD BIRTHDAY OPENAPI INITIATIVE

How to contribute to the OAS

Submit an issue on GitHub

Tyk.io ISA MuleSoft Intento ca technologies 42crunch

JOIN THE GROWING LIST OF OAI MEMBERS

Fonte:

<https://www.openapis.org/>

OpenAPI e Swagger



The screenshot shows the Swagger website homepage. At the top left is the Swagger logo with the text 'SMARTBEAR'. To the right are navigation links: 'Why Swagger >', 'Tools >', and 'Resources >'. Further right are two green buttons: 'Sign In' and 'Try Free'. The main heading 'Swagger' is in a large green font, followed by the tagline 'The Best APIs are Built with Swagger Tools'. Below this are two buttons: 'Try SwaggerHub' (solid green) and 'Explore Swagger Tools' (outlined green). The bottom section features five circular icons representing different API lifecycle stages: Design (notepad icon), Build (code with plus icon), Document (stack of papers icon), Test (code with checkmark icon), and Standardize (checklist icon). Each icon is accompanied by a title and a brief description of the stage.

SWAGGER
SMARTBEAR

Why Swagger > Tools > Resources > Sign In Try Free

Swagger

The Best APIs are Built with Swagger Tools

Try SwaggerHub Explore Swagger Tools

- Design**
Design and model APIs according to specification-based standards
- Build**
Build stable, reusable code for your API in almost any language
- Document**
Improve developer experience with interactive API documentation
- Test**
Perform simple functional tests on your APIs without overhead
- Standardize**
Set and enforce API style guidelines across your API architecture

Fonte: <https://swagger.io>

OpenAPI e Swagger

Podemos adicionar módulos que nos ajudam com a tarefa de documentar a nossa API

Um deles é o “swagger-ui-express” desenhado para funcionar com o middleware express de node.js

OpenAPI e Swagger

Antes de o utilizar devemos-o instalar com o comando

– “npm i swagger-ui-express --save”

Devemos no projeto anterior sobre a API REST de produtos adicionar ao ficheiro app.js

```
var swaggerUi = require('swagger-ui-express');  
var swaggerDocument = require('./swagger.json');
```

O ficheiro “swagger.json” será criado na raiz do projeto e serve para configurar os métodos da nossa api na documentação swagger

OpenAPI e Swagger

Devemos também adicionar o endereço onde a documentação sobre o nosso web servisse irá estar

```
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument));  
app.use('/api/v1', productsRouter);
```

Finalmente necessitamos apenas de configurar o ficheiro swagger.json

OpenAPI e Swagger

Começamos por
definir
informação
genérica sobre a
API

```
{
  "swagger": "2.0",
  "info": {
    "version": "1.0.0",
    "title": "Product API",
    "description": "REST API for products"
  },
  "host": "localhost:3000",
  "basePath": "/api/v1",
  "tags": [
    {
      "name": "Products",
      "description": "API for products in the system"
    }
  ],
  "schemes": [
    "http"
  ],
  "consumes": [
    "application/json"
  ],
  "produces": [
    "application/json"
  ],
}
```

OpenAPI e Swagger

Para cada endereço da nossa api é especificado

- o método http
- parametros
- resposta e modelo da resposta
- o que o serviços produz

```
"paths": {
  "/products": {
    "post": {
      "tags": [
        "Products"
      ],
      "description": "Create new user in system",
      "parameters": [
        {
          "name": "product",
          "in": "body",
          "description": "Product that we want to create",
          "schema": {
            "$ref": "#/definitions/Product"
          }
        }
      ],
      "produces": [
        "application/json"
      ],
      "responses": {
        "200": {
          "description": "New product is created",
          "schema": {
            "$ref": "#/definitions/Product"
          }
        }
      }
    }
  }
}
```

OpenAPI e Swagger

Para cada endereço da nossa api é especificado

- o método http
- parametros
- resposta e modelo da resposta
- o que o serviços produz

```
"get": {
  "tags": [
    "Products"
  ],
  "summary": "Get all products in system",
  "responses": {
    "200": {
      "description": "OK",
      "schema": {
        "$ref": "#/definitions/Products"
      }
    }
  }
},
"/product/{productId}": {
  "parameters": [
    {
      "name": "productId",
      "in": "path",
      "required": true,
      "description": "ID of product that we want to find",
      "type": "string"
    }
  ]
},
```

OpenAPI e Swagger

Para cada endereço da nossa api é especificado

- o método http
- parametros
- resposta e modelo da resposta
- o que o serviços produz

```
"get": {  
  "tags": [  
    "Products"  
  ],  
  "summary": "Get product with given ID",  
  "responses": {  
    "200": {  
      "description": "Product is found",  
      "schema": {  
        "$ref": "#/definitions/Product"  
      }  
    }  
  }  
},  
"delete": {  
  "summary": "Delete product with given ID",  
  "tags": [  
    "Products"  
  ],  
  "responses": {  
    "200": {  
      "description": "Product is deleted",  
      "schema": {  
        "$ref": "#/definitions/Product"  
      }  
    }  
  }  
},  
}
```

OpenAPI e Swagger

Para cada endereço da nossa api é especificado

- o método http
- parametros
- resposta e modelo da resposta
- o que o serviços produz

```
"put": {
  "summary": "Update product with give ID",
  "tags": [
    "Products"
  ],
  "parameters": [
    {
      "name": "product",
      "in": "body",
      "description": "Product with new values of properties",
      "schema": {
        "$ref": "#/definitions/Products"
      }
    }
  ],
  "responses": {
    "200": {
      "description": "Product is updated",
      "schema": {
        "$ref": "#/definitions/Product"
      }
    }
  }
},
},
},
},
```

OpenAPI e Swagger

É definido no
final também os
modelos de
dados usados
nos webservices

```
"definitions": {  
  "Product": {  
    "required": [  
      "name",  
      "_id"  
    ],  
    "properties": {  
      "_id": {  
        "type": "string",  
        "uniqueItems": true  
      },  
      "name": {  
        "type": "string",  
        "uniqueItems": true  
      },  
      "description": {  
        "type": "string"  
      },  
      "quantity": {  
        "type": "Number"  
      }  
    }  
  },  
  "Products": {  
    "type": "array",  
    "$ref": "#/definitions/Product"  
  }  
}
```

OpenAPI e Swagger

Após as alterações ao projeto, devemos ser agora capazes de usar a plataforma swagger para documentação teste da nossa api

Podemos consultar a informação através do endereço:

- <http://localhost:3000/api-docs>

OpenAPI e Swagger

Product API 1.0.0

[Base URL: localhost:3000/api/v1]

REST API for products

Schemes

HTTP

Products

API for products in the system

POST

/products

GET

/products

Get all products in system

GET

/product/{productId}

Get product with given ID

DELETE

/product/{productId}

Delete product with given ID

PUT

/product/{productId}

Update product with give ID

Models

OpenAPI e Swagger

Podemos invocar diretamente a partir do swagger os nossos serviços REST e obter o feedback sobre eles

Users API for users in the system

POST /users

Create new user in system

Parameters

Cancel

Name	Description
user	User that we want to create
(body)	Example Value Model <pre>{ "email": "new@new.pt", "lastName": "string", "firstName": "string"}</pre>

Cancel

OpenAPI e Swagger

Podemos invocar diretamente a partir do swagger os nossos serviços REST e obter o feedback sobre eles

The screenshot shows the Swagger UI interface for the `GET /users` endpoint. The interface is divided into several sections:

- Method and Path:** `GET /users` with the description "Get all users in system".
- Parameters:** A section labeled "Parameters" with a "Cancel" button. It indicates "No parameters".
- Execute/Cancel Buttons:** A blue "Execute" button and a "Clear" button.
- Responses:** A section labeled "Responses" with a "Response content type" dropdown set to "application/json".
- Curl:** A text area containing the curl command: `curl -X GET "http://localhost:3000/api/v1/users" -H "accept: application/json"`.
- Server response:** A section showing the response details. The "Code" is 200. The "Response body" is a JSON array containing one user object:

```
[ {  "_id": "5cabd14c740330028ba77df0",  "email": "new@new.pt",  "lastName": "string",  "firstName": "string",  "__v": 0 }
```

API Requests

Podemos também invocar web services de terceiros a partir de um backend

Para este efeito necessitamos de usar o módulo `https` e definir um callback para o sucesso ou insucesso da operação

Podemos encapsular chamadas a APIs de terceiros dentro das nossas APIs REST

API Requests

Exemplo de pedido GET
ao endereço
exemplo.com

```
const https = require('https')
const options = {
  hostname: 'exemplo.com',
  port: 443,
  path: '/todos',
  method: 'GET'
}

const req = https.request(options, (res) => {
  console.log(`statusCode: ${res.statusCode}`)

  res.on('data', (d) => {
    process.stdout.write(d)
  })
})

req.on('error', (error) => {
  console.error(error)
})

req.end()
```

API Requests

Exemplo de pedido POST
ao endereço
exemplo.com

Os métodos put e delete
também são possíveis
alterando o method no
objeto options

```
const https = require('https')
const data = JSON.stringify({
  item: 'item1'
})

const options = {
  hostname: 'exemplo.com',
  port: 443,
  path: '/item',
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Content-Length': data.length
  }
}

const req = https.request(options, (res) => {
  console.log(`statusCode: ${res.statusCode}`)

  res.on('data', (d) => {
    process.stdout.write(d)
  })
})

req.on('error', (error) => {
  console.error(error)
})

req.write(data)
req.end()
```

Referências

REST

- <https://developer.mozilla.org/en-US/docs/Glossary/REST>

Swagger and OpenAPI

- <https://github.com/swagger-api/swagger-node>
- <https://github.com/swagger-api/swagger-node/blob/master/docs/quick-start.md>

WebSockets

- <https://github.com/websockets/ws>
- <https://www.npmjs.com/package/websocket>

P.PORTO

REST APIs

Programação em Ambiente Web